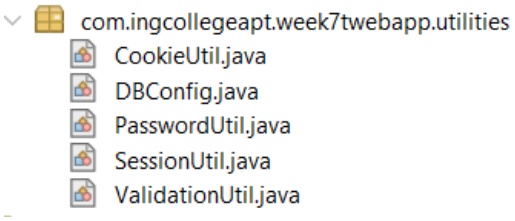
Week 7 Tutorial: Session and WebFilter

1. Unzip and open the week 7 starter project, clean build it.
2. Add the session and cookie utility files provided in util folder to the utilities package.

   2.1. Correct the package name according to the project

   2.2. Study the methods available on how to use Session for state management.



   2.3. For Session, we have methods for CRUD on session data in SessionUtil.java

   2.4. State management using Session.

   A. Store user info in session after successful log in.
   B. Invalidate user and session after log out.

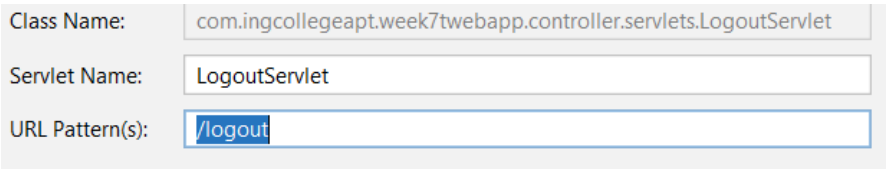
   2.5. In LoginServlet, after successful login store user data in session.

   >> if password match, add session store data.

```
if (matched) {
   // we store user object in session with user attribute
   SessionUtil.setAttribute(request, "user", user);
```

```
>> Check what above method does
```

   2.6. Create LogoutServlet so that we can invalidate the session and remove user data.

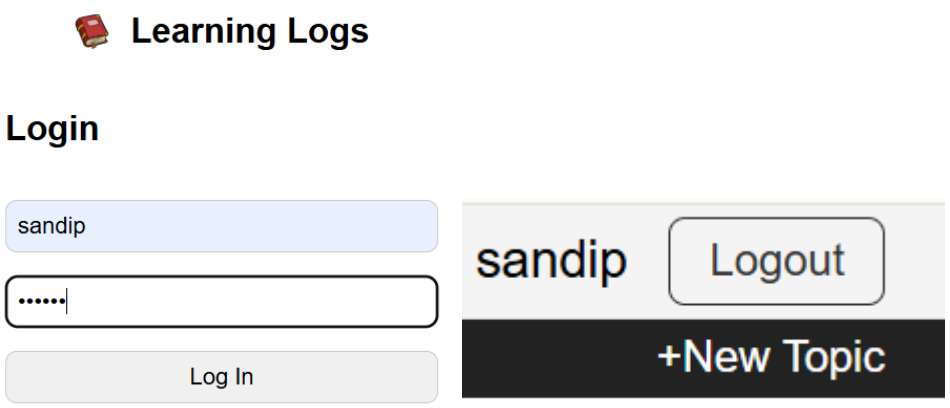


   2.7. For every page that has logout button, add following to create logout request and update username to show actual user name.

```
<div class="usersession">
    <h3>${sessionScope.user.getName()}</h3>
    <a href="logout" class="logout"
        onclick= "return confirm('Do you want to logout? ')"
   >Logout</a>
</div>
```

```
>>Do same in all:
   topiclist.jsp, topicadd.jsp, entrylist.jsp and entryadd.jsp
```

2.8. Run the login.jsp file, login with your credentials. If no user register one.



2.9. If we press logout, confirmation dialog appears and creates /logout path.

2.10. In LogoutServlet, handle this get request:

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
        // Invalidate the session that delete the user data from session
        SessionUtil.invalidateSession(request);

         // Now route to login page
        response.sendRedirect("login.jsp");
  }
```

2.11. Now, let us show topics and entries to the logged user only

A. In TopicServlet:

I. Change DAOs that require user data

```
//Updated for to include topics per user >> default section of doGet
User user = (User) SessionUtil.getAttribute(request, "user");
ArrayList<Topic> topics = tdao.fetchAllTopics(user.getId());

//insert topic to specific user >> add section of doPost
User user = (User) SessionUtil.getAttribute(request, "user");
boolean isAdded = tdao.insertTopic(topicName,user.getId());
```

B. In EntryServlet:

Check if the entry is for topic that the user is linked to

>> Workshop Task <<

3. Following web work flow is needed:

   A. If user is logged In:

      1. Login and Register Page is accessed >> route to topic page

      2. If other pages are accessed, allow them.

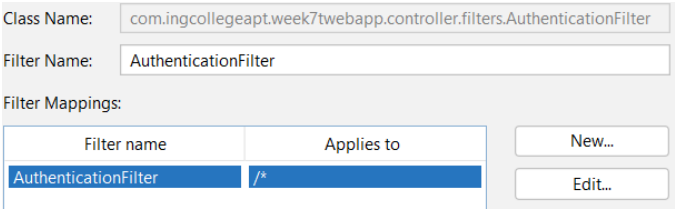
   B. If user is not logged In:

      1. Login and Register Page is accessed >> allow them.

      2. If other pages are accessed, route to login page.

3.2. We do this using Filter as it works on evey page defined through its url pattern.

3.3. Create Authorization filter in filter package:




```
@WebFilter(filterName = "AuthenticationFilter", urlPatterns = {"/*"})
public class AuthenticationFilter implements Filter {

    @Override
    public void init(FilterConfig filterConfig) throws ServletException {
        // Initialization logic, if required
    }

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
            FilterChain chain) throws IOException, ServletException {
    }

    @Override
    public void destroy() {
        // Cleanup logic, if required
    }
}
```

>> This Filter listens to all request as url pattern is “/*”

3.4. Add different pages and resources variables to track access, as instance variables.

```
private static final String LOGIN = "/login";
private static final String REGISTER = "/register";
private static final String HOME = "/topic";
private static final String ROOT = "/";
```

>> These are the default endpoints for out app.

3.5. In doFilter:

```
// Cast the request and response to HttpServletRequest and
HttpServletResponse
HttpServletRequest req = (HttpServletRequest) request;
HttpServletResponse res = (HttpServletResponse) response;
```

```
>> As request and response are ServletRequest and ServletResponse instance
   we downcast them to required Http objects.
```

```
// Get the requested URI
String uri = req.getRequestURI();
```

```
>> returns web app root and path: E.g. /AppName/login.jsp
>> From this String, we can check what type of resources are requested
```

```
        //All Static file links are also request, allow them
        if (uri.endsWith(".png") || uri.endsWith(".jpg") ||
uri.endsWith(".css")) {
            chain.doFilter(request, response);
            return;
        }
```

3.6. Let us handle the workflow defined above:

```
// Get the session and check if user is logged in
boolean isLoggedIn = SessionUtil.getAttribute(req, "user") != null;

        if (!isLoggedIn) {
            if (uri.endsWith(LOGIN) || uri.endsWith(REGISTER)) {
                chain.doFilter(request, response);
            } else {
                res.sendRedirect(req.getContextPath() + LOGIN);
            }
        } else {
            if (uri.endsWith(LOGIN) || uri.endsWith(REGISTER)) {
                res.sendRedirect(req.getContextPath() + HOME);
            } else {
                chain.doFilter(request, response);
            }
        }
```

```
>> Check if user is present in session; if user is logged in
>> If not logged:
     >> Check if login or register page and let it pass
     >> Else, call /login servlet
>> If logged in:
    >> Check for login or register page, call /topic
    >> Else, let it pass
```

3.7. Overall doFilter method:

```
    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
            FilterChain chain) throws IOException, ServletException {
        // Cast the request and response to HttpServletRequest and
HttpServletResponse
        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse res = (HttpServletResponse) response;

        // Get the requested URI
        String uri = req.getRequestURI();

        //Allow request for static files to pass in pages
       if (uri.endsWith(".png") || uri.endsWith(".jpg") ||
uri.endsWith(".css")) {
            chain.doFilter(request, response);
            return;
        }

        // Get the session and check if user is logged in
        boolean isLoggedIn = SessionUtil.getAttribute(req, "user") != null;
        if (!isLoggedIn) {
            if (uri.endsWith(LOGIN) || uri.endsWith(REGISTER)){
                chain.doFilter(request, response);
            } else {
                res.sendRedirect(req.getContextPath() + LOGIN);
            }
        } else {
            if (uri.endsWith(LOGIN) || uri.endsWith(REGISTER)){
                res.sendRedirect(req.getContextPath() + HOME);
            } else {
                chain.doFilter(request, response);
            }
        }
    }
```

3.8. LoginServlet will have issue with /login and /login.jsp.

>> Filter will **redirect** to /login when /login.jsp is dispatched from LoginServlet, this **get request** is not handled.

>> To solve this, in doGet of LoginServlet we will handle dispatch login.jsp. As dispatcher does not change uri, not checked by Filter. Add following in doGet, LoginServlet:

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
        RequestDispatcher rd = request.getRequestDispatcher("login.jsp");
        rd.forward(request, response);
    }
```

3.9. RegistetServlet will have same issue with /register and /register.

>> To solve this, Same as in doGet of LoginServlet, we define same in RegisterServlet

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
        RequestDispatcher rd = request.getRequestDispatcher("register.jsp");
        rd.forward(request, response);
    }
```

Learnings:

1. Session can store data in session scope and access this data via sessionScope in jsp and session object in servlets.
2. Filters intercept the request and response from client or browser as per defined in Filter url pattern.
3. chain.doFilter(): simply forwards the request and respond to the targeted endpoint or another Filter if available.

<< END OF SESSION: PLEASE COMPLETE AS ASSIGNMENT IF NOT COMPLETED IN CLASS>>